```
@RestController
@RequestMapping("books")
public class BookController {
  private static final Logger log =
    LoggerFactory.getLogger(BookController.class);      ❶
  private final BookService bookService;

  @GetMapping
  public Iterable<Book> get() {
    log.info(                                           ❷
      "Fetching the list of books in the catalog"
    );
    return bookService.viewBookList();
  }

  ...
}
```

❶ Defines a logger for the BookController class

❷ Logs the given message at the “info” level

NOTE Go ahead and define new loggers and log events for all the applications composing the Polar Bookshop system wherever it makes sense. As a reference, you can look at the source code repository accompanying this book (Chapter13/13-end).

The Mapped Diagnostic Context (MDC)

You’ll likely need to add common information to your log messages, such as the identifier of the currently authenticated user, the tenant for the current context, or the request URI. You could directly add that information to your log message, as you did in the previous listing, and it would work, but the data would not be structured. Instead, I prefer working with structured data.

SLF4J and common logging libraries, like Logback and Log4J2, support adding structured information depending on the request context (authentication, tenant, thread) through a tool named Mapped Diagnostic Context (MDC). If you’d like to know more about MDC, I recommend checking the official documentation for the specific logging library you’re using.

Now that our applications log messages as an event stream, we need to collect and store them in a central place that we can query. The following